Lab 0: Python Toolbox Setup

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

1 Overview **2**

2 Learning Objectives **2**

3 Pre-Lab: Environment Setup **2**
3.1 Step 1: Install Python Environment 2
3.2 Step 2: Verify Installation 3
3.3 Step 3: Pre-Lab Questions 3

4 Part 1: NumPy Basics for Robotics (30 min) **3**
4.1 1.1 Creating Matrices 3
4.2 1.2 Building Rotation Matrices Manually 4
4.3 1.3 Exercise 1: Verify SO(3) Properties 5

5 Part 2: Using spatialmath Library (30 min) **5**
5.1 2.1 Creating Rotations with SO3 5
5.2 2.2 Composing Rotations 6
5.3 2.3 Exercise 2: Order of Rotations 6

6 Part 3: Visualizing Coordinate Frames (30 min) **7**
6.1 3.1 Plotting a Single Frame 7
6.2 3.2 Animating a Rotation Sequence 7
6.3 3.3 Exercise 3: Visualize Rx, Ry, Rz 8

7 Part 4: Rotating Vectors (30 min) **9**
7.1 4.1 Transforming a Point 9
7.2 4.2 Exercise 4: Trace a Circle 9
7.3 4.3 Exercise 5: Verify Hand Calculations 10

8 Deliverables **10**
8.1 Required Files 10
8.2 Submission Format 11

9 Grading Rubric **11**

10 Additional Resources **11**

11 Appendix: Common Issues **11**
11.1 Plot window doesn't appear 11
11.2 "Module not found" error 12
11.3 Rotation looks wrong 12

1 Overview

In this lab, you will set up your Python development environment and learn the fundamental tools for robotics simulation. By the end, you'll be able to create rotation matrices, visualize 3D coordinate frames, and verify mathematical operations programmatically.

Duration: 2 hours

Software Required:

- Python 3.8+ with Anaconda or virtual environment
- Libraries: `numpy`, `matplotlib`, `spatialmath-python`, `roboticstoolbox-python`

2 Learning Objectives

By the end of this lab, you will be able to:

1. Set up a Python environment for robotics simulation
2. Create and manipulate rotation matrices using NumPy
3. Use the `spatialmath` library for 3D transformations
4. Visualize coordinate frames in 3D
5. Verify hand calculations with Python code

3 Pre-Lab: Environment Setup

Complete these steps **before** coming to lab.

3.1 Step 1: Install Python Environment

Option A - Anaconda (Recommended):

```
# Download from anaconda.com/download, then:
conda create -n robotics python=3.10
conda activate robotics
pip install roboticstoolbox-python spatialmath-python numpy matplotlib
```

Option B - Virtual Environment:

```
python -m venv robotics_env
source robotics_env/bin/activate  # Windows: robotics_env\Scripts\activate
pip install roboticstoolbox-python spatialmath-python numpy matplotlib
```

3.2 Step 2: Verify Installation

Create a file called `test_install.py`:

```
import numpy as np
from spatialmath import SO3, SE3
import roboticstoolbox as rtb
import matplotlib.pyplot as plt

print("All imports successful!")
print(f"NumPy version: {np.__version__}")
print(f"Robotics Toolbox version: {rtb.__version__}")

# Quick test
R = SO3.Rz(45, 'deg')
print(f"\nRotation matrix Rz(45°):\n{R}")
```

Run it:

```
python test_install.py
```

If you see the rotation matrix printed without errors, you're ready!

3.3 Step 3: Pre-Lab Questions

Answer these questions before lab (write in your lab notebook or document):

1. What is a rotation matrix? How many elements does it have?
2. What does $SO(3)$ mean mathematically?
3. Write out the formula for $R_z(45°)$ by hand.

---

4 Part 1: NumPy Basics for Robotics (30 min)

4.1 1.1 Creating Matrices

Open a new Python file or Jupyter notebook. Start with basic matrix operations:

```
import numpy as np

# Create a 3x3 identity matrix
I = np.eye(3)
```

```
print("Identity matrix:")
print(I)

# Create a vector
v = np.array([1, 0, 0])
print(f"\nVector v = {v}")

# Matrix-vector multiplication
result = I @ v  # @ is matrix multiplication in Python
print(f"I @ v = {result}")
```

4.2 1.2 Building Rotation Matrices Manually

Create the elementary rotation matrices from scratch:

```
import numpy as np

def Rz(theta_deg):
    """Rotation about z-axis by theta degrees"""
    theta = np.radians(theta_deg)
    c, s = np.cos(theta), np.sin(theta)
    return np.array([
        [c, -s, 0],
        [s,  c, 0],
        [0,  0, 1]
    ])

def Ry(theta_deg):
    """Rotation about y-axis by theta degrees"""
    theta = np.radians(theta_deg)
    c, s = np.cos(theta), np.sin(theta)
    return np.array([
        [ c, 0, s],
        [ 0, 1, 0],
        [-s, 0, c]
    ])

def Rx(theta_deg):
    """Rotation about x-axis by theta degrees"""
    theta = np.radians(theta_deg)
    c, s = np.cos(theta), np.sin(theta)
    return np.array([
        [1, 0,  0],
        [0, c, -s],
        [0, s,  c]
```

```
    ])

# Test: Rz(90°)
R = Rz(90)
print("Rz(90°) =")
print(R)
```

4.3 1.3 Exercise 1: Verify SO(3) Properties

Task: Write code to verify that your rotation matrix is in $SO(3)$.

```
def is_SO3(R, tol=1e-6):
    """Check if R is a valid rotation matrix"""
    # Check 1: R^T @ R = I
    check1 = np.allclose(R.T @ R, np.eye(3), atol=tol)

    # Check 2: det(R) = +1
    check2 = np.isclose(np.linalg.det(R), 1.0, atol=tol)

    return check1 and check2

# Test your Rz matrix
R = Rz(45)
print(f"Is Rz(45°) in SO(3)? {is_SO3(R)}")

# Test with a non-rotation matrix
bad_matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"Is bad_matrix in SO(3)? {is_SO3(bad_matrix)}")
```

Record your results: What did you get for each test?

---

5 Part 2: Using spatialmath Library (30 min)

5.1 2.1 Creating Rotations with SO3

The `spatialmath` library provides a cleaner interface:

```
from spatialmath import SO3
import numpy as np

# Create rotation matrices
Rx = SO3.Rx(30, 'deg')
Ry = SO3.Ry(45, 'deg')
Rz = SO3.Rz(90, 'deg')
```

```
print("Rz(90°) using spatialmath:")
print(Rz)

# Access the underlying numpy array
R_numpy = Rz.R  # or Rz.A for the full array
print(f"\nAs numpy array:\n{R_numpy}")
```

5.2 2.2 Composing Rotations

```
from spatialmath import SO3

# Compose rotations
R1 = SO3.Rz(90, 'deg')
R2 = SO3.Rx(45, 'deg')

# Method 1: Using * operator
R_combined = R1 * R2
print("R1 * R2 =")
print(R_combined)

# Method 2: Explicit matrix multiplication
R_combined_np = R1.R @ R2.R
print("\nSame result using numpy:")
print(R_combined_np)
```

5.3 2.3 Exercise 2: Order of Rotations

Task: Verify that rotation order matters.

```
from spatialmath import SO3

R1 = SO3.Rz(90, 'deg')
R2 = SO3.Rx(90, 'deg')

# Compute both orders
R_12 = R1 * R2
R_21 = R2 * R1

print("R1 * R2 =")
print(R_12)
print("\nR2 * R1 =")
print(R_21)

# Are they equal?
```

```
print(f"\nAre they equal? {np.allclose(R_12.R, R_21.R)}")
```

Question: What is the physical interpretation of this result?

---

6 Part 3: Visualizing Coordinate Frames (30 min)

6.1 3.1 Plotting a Single Frame

```
from spatialmath import SO3
import matplotlib.pyplot as plt

# Create a rotation
R = SO3.Rz(45, 'deg')

# Plot the rotated frame
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')

# Plot identity frame (world)
SO3().plot(ax=ax, frame='W', color='black', length=1)

# Plot rotated frame
R.plot(ax=ax, frame='A', color='red', length=1)

ax.set_xlim([-1.5, 1.5])
ax.set_ylim([-1.5, 1.5])
ax.set_zlim([-1.5, 1.5])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('World Frame (black) and Rotated Frame (red)')

plt.show()
```

6.2 3.2 Animating a Rotation Sequence

```
from spatialmath import SO3
import matplotlib.pyplot as plt
import numpy as np

# Create figure
```

```
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Plot world frame
SO3().plot(ax=ax, frame='W', color='black', length=1)

# Plot multiple rotated frames
colors = ['red', 'green', 'blue', 'orange']
angles = [0, 30, 60, 90]

for angle, color in zip(angles, colors):
    R = SO3.Rz(angle, 'deg')
    R.plot(ax=ax, frame=f'{angle}°', color=color, length=0.8)

ax.set_xlim([-1.5, 1.5])
ax.set_ylim([-1.5, 1.5])
ax.set_zlim([-1.5, 1.5])
ax.set_title('Rotation about Z-axis: 0°, 30°, 60°, 90°')

plt.show()
```

6.3 3.3 Exercise 3: Visualize Rx, Ry, Rz

Task: Create a figure with 3 subplots showing:

1. Rotation about X-axis by 45°
2. Rotation about Y-axis by 45°
3. Rotation about Z-axis by 45°

Each subplot should show the world frame (black) and rotated frame (red).

```
from spatialmath import SO3
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(15, 5))

rotations = [
    (SO3.Rx(45, 'deg'), 'Rx(45°)'),
    (SO3.Ry(45, 'deg'), 'Ry(45°)'),
    (SO3.Rz(45, 'deg'), 'Rz(45°)')
]

for i, (R, title) in enumerate(rotations):
    ax = fig.add_subplot(1, 3, i+1, projection='3d')

    # YOUR CODE HERE: Plot world frame and rotated frame
```

```
    # ...

    ax.set_title(title)
    ax.set_xlim([-1.5, 1.5])
    ax.set_ylim([-1.5, 1.5])
    ax.set_zlim([-1.5, 1.5])

plt.tight_layout()
plt.show()
```

Save your figure as `rotation_comparison.png`

---

7 Part 4: Rotating Vectors (30 min)

7.1 4.1 Transforming a Point

```
from spatialmath import SO3
import numpy as np

# Define a point in 3D
p = np.array([1, 0, 0])  # Point on x-axis

# Rotate by 90° about z-axis
R = SO3.Rz(90, 'deg')

# Transform the point
p_rotated = R * p

print(f"Original point: {p}")
print(f"After Rz(90°): {p_rotated}")
```

7.2 4.2 Exercise 4: Trace a Circle

Task: Use rotation to trace out points on a circle.

```
from spatialmath import SO3
import numpy as np
import matplotlib.pyplot as plt

# Start with a point on the x-axis
p0 = np.array([1, 0, 0])

# Collect rotated points
points = []
```

```python
for angle in range(0, 361, 10):
    R = SO3.Rz(angle, 'deg')
    p_rotated = R * p0
    points.append(p_rotated)

points = np.array(points)

# Plot
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')

ax.plot(points[:, 0], points[:, 1], points[:, 2], 'b-', linewidth=2)
ax.scatter([1], [0], [0], color='red', s=100, label='Start')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Circle traced by rotating a point about Z-axis')
ax.legend()

plt.show()
```

7.3 4.3 Exercise 5: Verify Hand Calculations

Task: Verify the following calculation using Python:

Given $R_z(30°)$ and $p = [1, 0, 0]^T$, compute $R_z(30°) \cdot p$.

1. First, calculate by hand (show your work)
2. Then verify with Python

Expected result: $[\cos(30°), \sin(30°), 0]^T = [0.866, 0.5, 0]^T$

---

8 Deliverables

Submit the following by the deadline:

8.1 Required Files

1. **`lab0_exercises.py`** - Python file containing:
   - Your `Rx`, `Ry`, `Rz` functions
   - Your `is_SO3` function
   - Solutions to Exercises 1-5
2. **`rotation_comparison.png`** - Figure from Exercise 3

3. `lab0_report.pdf` - Short report (1-2 pages) containing:
    - Answers to Pre-Lab questions
    - Screenshots of your working code
    - Answer to Exercise 2 question (physical interpretation)
    - Any difficulties encountered and how you solved them

8.2 Submission Format

- Combine all files into a ZIP archive: `Lab0_YourName.zip`
- Submit via course portal

---

9 Grading Rubric

Component	Points
Pre-Lab questions answered	10
Exercise 1: SO(3) verification	15
Exercise 2: Rotation order	15
Exercise 3: Visualization (3 subplots)	20
Exercise 4: Circle tracing	15
Exercise 5: Hand calculation + verification	15
Code quality and comments	10
Total	**100**

---

10 Additional Resources

- **spatialmath documentation:** petercorke.github.io/spatialmath-python
- **NumPy quickstart:** numpy.org/doc/stable/user/quickstart.html
- **Matplotlib 3D plotting:** matplotlib.org/stable/gallery/mplot3d

---

11 Appendix: Common Issues

11.1 Plot window doesn't appear

Try adding this at the start of your script:

```
import matplotlib
matplotlib.use('TkAgg')  # or 'Qt5Agg'
```

11.2 "Module not found" error

Make sure your environment is activated:

```
conda activate robotics
```

11.3 Rotation looks wrong

Remember: Python trigonometric functions use **radians**, not degrees!

```
np.cos(90)        # WRONG - this is 90 radians!
np.cos(np.pi/2)   # Correct - this is 90 degrees in radians
```

Or use the `'deg'` argument in spatialmath:

```
SO3.Rz(90, 'deg')  # Correct!
```